

Universidad Internacional del Ecuador



Sistema de Monitoreo de índice UV en tiempo real con alertas automáticas

Integrantes:

- Manuel Alejandro
- Gabriel Rios
- Juan Campana
- Daddy Acacho

Dominio:

IoT aplicado al monitoreo ambiental

Fecha:

22 de mayo de 2026

Desarrollo de investigación

¿Qué limitaciones reales de UML intentó resolver Simon Brown con C4?

UML fue pensado para ser exhaustivo, lo que en la práctica lo convirtió en una herramienta difícil de adoptar. Sus diagramas requieren conocer notaciones formales complejas, mezclan diferentes niveles de abstracción sin distinguirlos claramente, y generan artefactos tan densos que solo otros ingenieros los entienden. Simon Brown identificó que el problema de fondo no era la notación en sí, sino la falta de niveles de zoom explícitos: un arquitecto necesita diferentes vistas para distintas audiencias

Explica brevemente los 4 niveles y qué pregunta fundamental responde cada uno.

Nivel 1 Contexto: muestra el sistema como una caja negra dentro de su entorno. Responde a la pregunta ¿qué hace el sistema y con quién interactúa? Es la vista para stakeholders no técnicos: aparecen los usuarios, los sistemas externos y las relaciones entre ellos, sin ningún detalle de implementación. En UV Shield, este nivel muestra el sistema completo junto a los ciudadanos, el administrador y servicios como Telegram Bot API.

Nivel 2 Contenedores: abre la caja negra y muestra las partes desplegables o ejecutables que la componen. Responde a ¿en qué piezas se divide el sistema y cómo se comunican? Un contenedor puede ser una aplicación web, una API, una base de datos o un firmware. En UV Shield serían el firmware del ESP8266, el backend Node.js/Express, la base de datos SQLite y la PWA.

Nivel 3 Componentes: entra dentro de un contenedor específico y muestra sus módulos lógicos internos. Responde a ¿qué responsabilidades tiene cada parte del contenedor? Por ejemplo, dentro del backend de UV Shield estarían el módulo de ingestión de lecturas, el servicio de alertas, la lógica de umbrales UV y el registro de suscripciones push.

Nivel 4 Código: es el nivel de mayor detalle y muestra cómo se implementa un componente en clases, funciones o interfaces. Responde a ¿cómo está construido internamente este componente? Es opcional y generalmente se genera desde el propio código fuente. En UV Shield correspondería, por ejemplo, al diagrama de clases de la función que calcula el índice UV a partir de los valores UVA y UVB del sensor.

¿Por qué el Nivel 1 (Diagrama de Contexto) es vital para los stakeholders no técnicos?

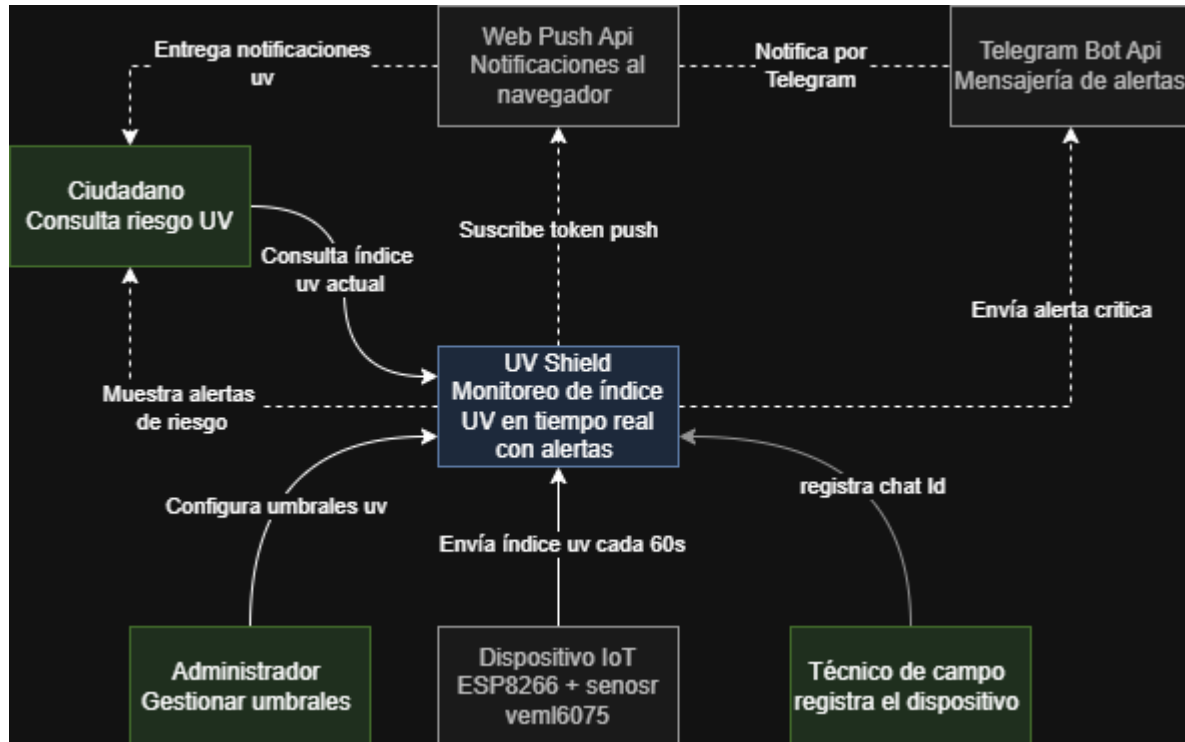
El Diagrama de Contexto elimina todo detalle de implementación y se enfoca únicamente en quiénes usan el sistema y con qué otro sistema se relaciona. Un gerente, cliente o autoridad universitaria puede leer este diagrama sin conocimientos de programación y entender de inmediato el alcance del proyecto, sus fronteras y sus dependencias externas. Es la única vista que permite una conversación estratégica sobre el sistema sin que la discusión se desvíe hacia detalles técnicos. En proyectos de IoT como UV Shield, este nivel también clarifica responsabilidades: qué parte es nuestra, qué parte es de terceros (Telegram, Web Push), y qué datos cruzan esas fronteras.

Relación entre un "contenedor" del C4 Model y el concepto de microservicio.

En C4, un contenedor es cualquier unidad que se ejecuta o despliega de forma independiente: una aplicación web, una API, una base de datos, un firmware embebido. Un microservicio es esencialmente un contenedor con responsabilidad única, despliegue independiente e interfaz bien definida. La relación es que todo microservicio es un contenedor C4, pero no todo contenedor es un microservicio. En UV

Shield, el firmware del ESP8266, el backend Node.js y la PWA son tres contenedores C4 distintos; si el backend se dividiera en un servicio de ingestión y otro de notificaciones independientes, cada uno sería tanto un contenedor C4 como un microservicio.

Diagrama de Contexto



Leyenda y descripción de sistemas externos

Se eligió Telegram Bot API porque permite entregar alertas críticas directamente al celular del usuario sin requerir una app nativa, usando únicamente un chat ID registrado desde el formulario web. Web Push API se incluyó como canal complementario para notificar al navegador incluso con la PWA cerrada, sin depender de cuentas externas adicionales. El Dispositivo IoT se modela como sistema externo porque opera de forma física e independiente en campo, comunicándose con el backend cada 60 segundos vía HTTP POST. Se descartó el correo electrónico como canal de alertas porque su latencia de entrega no garantiza la inmediatez necesaria cuando el índice UV representa un riesgo real para la salud del ciudadano.

Uso de IA en esta actividad

La IA fue útil para reformular las definiciones teóricas de C4 en un lenguaje más preciso y libre de jerga innecesaria, especialmente en la comparación con UML. La IA propuso inicialmente incluir un servicio de correo electrónico (SMTP) como canal de alertas, pero el grupo lo descartó porque nuestro anteproyecto ya justifica técnicamente la superioridad de Telegram para eventos UV críticos por su inmediatez.

Enlace de githb: <https://github.com/gariosfe/doc-proyecto.git>